Design and Implementation of 5-Stage Pipelined RISC-V Processor on FPGA

Pankaj Nair V. M.
Dept. of Electronics and Communication Engineering
College of Engineering Trivandrum
APJ Abdul Kalam Technological University
Trivandrum, India
pankajnair01@gmail.com

Lalu V.
Dept. of Electronics and Communication Engineering
College of Engineering Trivandrum
APJ Abdul Kalam Technological University
Trivandrum, India
laluv@cet.ac.in

2024 28th International Symposium on VLSI Design and Test (VDAT) | 979-8-3503-8010-1/24/$31.00 ©2024 IEEE | DOI: 10.1109/VDAT63601.2024.10705665

***Abstract*—This paper presents the design and implementation of a 32-bit, in-order, 5-stage pipelined RISC-V processor, executed on the Basys 3 Artix-7 FPGA. RISC-V, an open-source Instruction Set Architecture (ISA), has gained significant attention due to its simplicity, scalability, and modularity. Our focus is on implementing a processor core adhering to the RV32IM ISA, which incorporates static branch prediction and a hybrid encoding approach using both one-hot and binary encoding schemes. The primary motivation is to develop a RISC-V processor that is not only accessible for beginners but also sufficiently lightweight for deployment on modest FPGA platforms. The processor integrates a Universal Asynchronous Receiver Transmitter (UART) peripheral, designed using a finite state machine (FSM), transforming it into a basic System-on-Chip (SoC). The design and implementation were carried out using SystemVerilog within the Vivado Design Suite 2023.2. Functional verification was conducted through simulation using the Vivado Simulator, and further validation was achieved by monitoring output data on a serial terminal post-FPGA implementation. The processor achieved a maximum operating frequency of 65 MHz and consumed a total chip power of 119 mW. This paper provides a comprehensive overview of the literature, details the design methodology, describes the experimental setup, and offers a comparative evaluation with similar processors.**

***Keywords*—RISC-V, FPGA, Pipeline, Microarchitecture, RTL**

I. Introduction

The landscape of processor architectures is undergoing continuous transformation, driven by the relentless demand for enhanced performance and efficiency. In recent years, Reduced Instruction Set Computing (RISC) processors have gained substantial traction due to their emphasis on a streamlined set of instructions that optimize performance. Among the various RISC architectures, RISC-V has emerged as a notable contender due to its unique advantages and open-source nature. Originally developed by the University of California, Berkeley, RISC-V was intended to support research and education in computer architecture. However, it has since evolved into a highly promising open-source Instruction Set Architecture (ISA) with the potential for widespread adoption in academic and industrial domains. [1] provides a comprehensive overview of the RISC-V architecture and helps provide a solid foundation, covering its history, advantages, and other key aspects.

The RISC-V Foundation was established in 2015 to oversee the maintenance, development, and dissemination of RISC-V's intellectual property. The foundation also ensures the provision of a comprehensive and compatible toolchain, which is crucial for a seamless development process and broad adoption of the ISA [2]. Recent developments in the RISC-V ecosystem include enhanced support for vector extensions, which are crucial for advanced data processing tasks, and the introduction of the RISC-V Hypervisor extension, which enables efficient virtualization, broadening its applicability in modern computing environments. Security features such as the Platform Management Unit (PMP) have been integrated to enhance memory access security in modern System-on-Chip (SoC) designs. RISC-V's extensibility makes it attractive for building custom accelerators for AI and Machine Learning (ML) tasks, enabling hardware and software co-design for improved performance.

Several open-source processors based on RISC-V have been developed, including The Berkeley Out of Order Machine (BOOM), SiFive's Freedom platforms (E300 and U500), and the Shakti processor family from IIT Madras. These processors typically employ complex pipelined architectures to achieve high performance while minimizing hardware overhead. Despite their power, these intricate designs challenge enthusiasts and learners aiming to grasp the fundamental principles of the RISC-V ISA.

Addressing this gap, this paper presents the design and implementation of a 32-bit RISC-V processor core that is both lightweight and beginner-friendly. The design of the processor was guided by a focus on simplicity and modularity. The processor features a single-core, in-order, 5-stage pipeline architecture, which balances performance with ease of understanding. The design consists of three main components: the Datapath, the Control Logic, and the Hazard Unit. The Datapath handles the flow of data through the processor, executing instructions and performing arithmetic operations in 5 stages: Instruction Fetch, Instruction Decode, Execute, Memory Access, and Write Back. The Control Logic orchestrates the operation of the Datapath, generating control signals that manage the execution of instructions. It uses both binary and one-hot encoding to achieve an optimal balance between performance and resource utilization. The Hazard Unit is responsible for detecting and resolving hazards that can

979-8-3503-8010-1/24/$31.00 ©2024 IEEE

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:53:30 UTC from IEEE Xplore. Restrictions apply.

occur during instruction execution, specifically data hazards and control hazards, ensuring smooth pipeline operation. A key component of this design is the integration of a Universal Asynchronous Receiver Transmitter (UART) module, transforming the processor into a basic System-on-Chip (SoC). This inclusion facilitates hardware verification and practical deployment in real-world applications.

This paper is structured as follows: Section II reviews related work in the literature, Section III provides an overview of the RISC-V ISA, Section IV details the design methodology, Section V discusses the experimental setup, and Section VI presents the results. The paper concludes in Section VII with remarks on future work.

II. Literature Review

[3] presents a soft-core 32-bit RISC-V processor design that expands the base integer instruction set with multiplication and division capabilities. The authors propose two pipelined implementations – three-stage and five-stage – to explore the trade-off between performance and resource utilization. A key innovation is the hierarchical decoding technique, which reduces resource consumption in the decoding stage. Additionally, the authors use one-hot encoding for the control signals which helped reduce logic and increase the frequency of the core.

[4] proposes a RISC-V processor targeting a reduced RV32IM instruction set for efficiency. It implements a three-stage pipeline with dedicated fetch, decode, and execute modules to accelerate instruction processing. Static branch prediction and a Harvard memory structure are employed to further enhance performance. The authors mitigate data hazards by utilizing stalling and forwarding techniques within the pipeline to ensure smooth instruction execution.

[5] presents the design of a 32-bit RISC-V processor core targeting educational purposes in the field of embedded systems. The authors aim to bridge the gap between theoretical knowledge and practical application by offering a simple and understandable architecture The proposed model provides a comprehensive learning environment for embedded system concepts using RISC-V processors and assembly language programming.

[6] demonstrates the potential of flexible architectures over fixed ones. The authors emphasize the benefits of flexibility in balancing computational power and resource allocation according to application constraints and target hardware through their core, AsteRISC. The flexibility of the datapath allows for the addition of co-processors and sequencing operations over multiple cycles without compromising performance. The proposed configurable RISC-V processor architecture strikes a balance between clock cycles per instruction, maximum operating frequency, resource utilization, and power consumption, making it adaptable to specific application constraints on both FPGA and ASIC targets.

[7] presents a 32-bit RISC-V pipelined processor design that addresses data and control hazards to enhance performance. The authors propose a five-stage pipeline implementation to fully execute the RV32I ISA. Key innovations include data forwarding and branch prediction techniques to eliminate hazards, thus maintaining pipeline efficiency. Data forwarding, implemented through IE-IE and Mem-IE techniques, provides data to dependent instructions before the write-back stage, reducing delays. The use of a two-phase clock in Mem-IE forwarding helps avoid pipeline stalls.

[8] presents the implementation of a five-stage pipelined soft processor core using the RV32I ISA. The proposed processor core executes all RV32I instructions, except FENCE, ECALL, and CSR. The five-stage pipelined design offers a 37% speed improvement over the author's previous single-cycle design. Despite the increase in throughput, the critical path—comprising the MEM/WB register, ALU multiplexer, ALU, branching unit, PC multiplexer, and PC—presents a bottleneck. Balancing this long combinational path could further enhance the maximum frequency of the processor core.

[9] describes the design and implementation of a UART peripheral for serial data communication using FPGA. The authors follow a modular structure to implement the UART module. The transmitter generates a 4 Mbps baud rate from a system clock using a dedicated baud rate generator module. The core of the transmitter is a finite state machine (FSM) that manages the data transmission process across four states: IDLE, LOAD, SHIFT, and WAIT. A parallel-in serial-out (PISO) register receives parallel data, adds framing bits and transmits the data serially using the baud clock.

III. RISC-V RV32IM Instruction Set

RISC-V is a load-store ISA based on the principles of Reduced Instruction Set Computing (RISC). This architecture separates data processing instructions from memory access instructions, simplifying the design and enhancing the performance. A key strength of RISC-V lies in its modularity. The ISA is divided into privilege levels and optional extensions, allowing for customization of processor cores based on specific application needs. To balance simplicity and flexibility, RISC-V utilizes a compact set of 6 instruction formats as shown in Table I.

TABLE I
Core Instructions Format

31-25	24-20	19-15	14-12	11-7	0-6	Bits/Format
funct7	rs2	rs1	funct3	rd	opcode	R-Type
imm[11:0]		rs1	funct3	rd	opcode	I-Type
imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	S-Type
imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode	B-Type
imm[31:12]				rd	opcode	U-Type
imm[20|10:1|11|19:12]				rd	opcode	J-Type

The RISC-V ISA allows for data widths of 32-bit, 64-bit, and 128-bit. This paper implements a processor with a data width of 32-bit. The RV32I (Base Integer), mandatory for any RISC-V implementation, provides core instructions encompassing loads, stores, control flow mechanisms, and integer computations. The RV32I base instruction set acts as a foundation, and optional extensions empower it with additional capabilities, unlocking the potential for the processor core

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:53:30 UTC from IEEE Xplore. Restrictions apply.

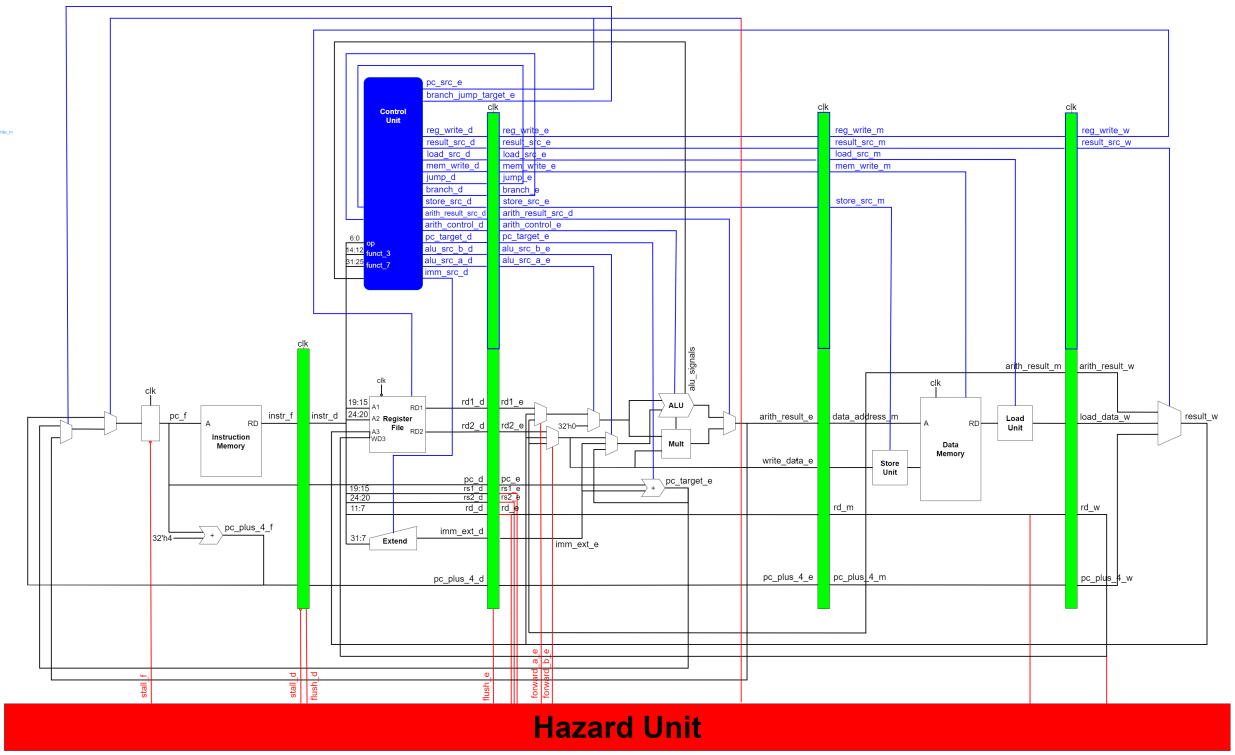


Fig. 1. Microarchitecture of the Processor

to address a wider range of applications. [10] provides a comprehensive discussion of the RISC-V ISA.

RISC-V employs a register file with 32 registers. One register (index 0) is pre-wired to always contain zero, while the remaining 31 registers (indices 1 to 31) serve as general-purpose registers for computations. For the 32-bit system, the width of all general-purpose registers is 32-bit.

IV. Methodology

This section provides an overview of the design aspects and architecture of the processor. The processor is designed using the Vivado Design Suite 2023.2. It is implemented with a 5-stage pipelined microarchitecture as seen in Fig. 1, consisting of the following stages: (a) Instruction Fetch, (b) Instruction Decode, (c) Execute, (d) Memory Access, and (e) Write Back. All stages of the processor pipeline are in order. The subsequent subsections describe the top level and microarchitecture of this design in detail.

A. Top Level Overview

The core of the design revolves around three central components working in tandem: Processor Core, Instruction Memory, and Data Memory as shown in Fig. 2. Data flows between these components through dedicated data lines. A specific line, labeled *instr_f* in the design, ferries instructions from Instruction Memory to the Processor Core. Likewise, another line, *read_data_m*, facilitates data transfer from Data Memory to the Processor Core. This coordinated interplay between fetching instructions, retrieving data, and executing operations is the essence of how the design performs computations.

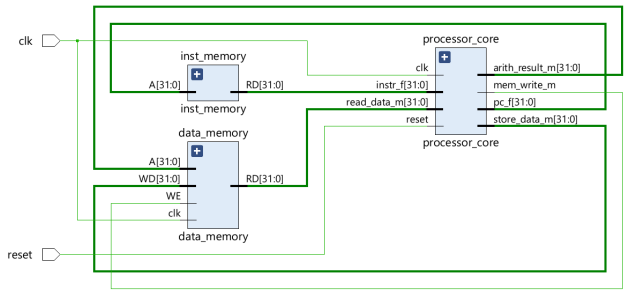


Fig. 2. Top RTL Schematic of the Processor

B. Datapath

The Datapath acts as the processor's workhorse, handling instruction execution. It fetches instructions, prepares operands, performs operations in the ALU and Multiply Unit, accesses memory if needed, and writes results back. It interacts with the Hazard Unit for smooth operation and receives control signals from the Control Unit to orchestrate the entire flow.

1) Fetch Stage: The fetch stage acts as the initiator, retrieving instructions from the Instruction Memory. It pinpoints the location of the next instruction based on the Program Counter (PC) and fetches the instruction code from memory. A dedicated adder is also included for incrementing the PC in each cycle.

2) Decode Stage: The decode stage acts as the interpreter, dissecting the fetched instruction and extracting critical in-

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:53:30 UTC from IEEE Xplore. Restrictions apply.

formation. The Register File has 31 general-purpose registers accessed to retrieve operands required by the instruction The Extend Unit plays a crucial role, particularly for instructions involving immediate values. It provides the properly sign extended and reordered 32-bit immediate operand for the ALU as per Table I.

3) Execute Stage: The execute stage serves as the powerhouse of instruction processing, where heavy computational work takes place, supported by specialized units. The Arithmetic Logic Unit (ALU) handles a broad spectrum of arithmetic, logical, and bitwise operations, while the Multiply Unit specializes in multiplication tasks. For branch instructions, the ALU calculates the condition and provides the results to the Control Unit which then decides whether a branch is to be taken. The stage also has a dedicated adder for calculating target addresses for branch or jump instructions.

4) Memory Stage: The memory stage acts as a crucial link between the Processor Core and Data Memory, facilitating the exchange of essential data for computations. Data is stored or loaded from the memory as instructed by the decoded information. The Store Unit manages store operations, facilitating the writing of data from the Register File to the Data Memory. The Load Unit manages load operations, retrieving the data word from Data Memory and transferring it to the Register File.

The Load Unit and Store Unit determine the byte/half-word offset within a word using specific bits in the memory address as shown in Table II.

TABLE II
BYTE AND HALF-WORD OFFSETS

Address	Offset
Byte	
address[1:0] = 2'b00	data[8:0]
address[1:0] = 2'b01	data[15:9]
address[1:0] = 2'b10	data[23:16]
address[1:0] = 2'b11	data[31:24]
Half-Word	
address[0] = 1'b0	data[15:0]
address[0] = 1'b1	data[31:16]

5) Write-Back Stage: The write-back stage acts as the finalizer, responsible for storing the results of the operation back to the designated register in the Register File. This ensures that the outcome of the instruction is readily available for subsequent instructions to utilize.

C. Control Unit

The Control Unit coordinates the execution of instructions within a processor. It acts as the brain of the processor, interpreting incoming instructions and directing other functional units to perform the necessary operations. This section delves into the key components of the control logic and their functionalities.

1) Main Decoder: The Control Unit receives instructions as encoded bit patterns as shown in Table I. The Main Decoder is responsible for unraveling these instructions. It analyzes the opcode field within the instruction, which specifies the type of operation to be performed. Based on the decoded opcode, the Main Decoder generates various control signals that steer the subsequent stages of the control logic.

2) ALU Decoder: The ALU Decoder takes the output of the main decoder, specifically the *arith_op* signal, along with the *funct_7* and *funct_3* fields from the instruction. These additional fields provide more granular details about the specific arithmetic or logical operation to be carried out. By analyzing this combined information, the ALU decoder generates control signals for the ALU and the Multiply Unit.

3) Load-Store Unit: The Load-Store Unit handles the transfer of data between the processor core and memory. It relies on the *funct_3* field of the instruction to determine the type of load or store operation required. Based on this information, the Load-Store Unit generates control signals for the Load Unit and the Store Unit. The control signals are one-hot encoded, as suggested in [3], to reduce the logic which increased the operating frequency from 62 Mhz to 65 MHz.

4) Branch Unit: Conditional branching instructions alter the normal execution sequence of a program. The Branch Unit is responsible for evaluating whether or not to take a branch based on the outcome of previous operations. It receives signals from the ALU, namely zero, negative, carry, and overflow, which indicate the results of the last computation. By analyzing these signals and comparing them with the branching condition specified in the instruction, the Branch Unit determines whether to alter the Program Counter and jump to a different instruction address.

D. Hazard Unit

In a pipelined system, simultaneous handling of multiple instructions can lead to hazards when dependencies arise between instructions. Read After Write (RAW) data hazards occur when an instruction relies on a result from another instruction that has not yet been written to the Register File. This is resolved by forwarding or stalling the pipeline until the result is available, as described in [7]. This design employs static branch prediction, where the branches are predicted as not taken.

Forwarding entails the Hazard Unit computing control signals for forwarding multiplexers, which select operands from the Register File, the *alu_result_m* signal in the Memory Stage, or the *result_w* signal in the Writeback stage.

Stalling is facilitated by enabling inputs in the Fetch and Decode pipeline registers, along with a synchronous reset input in the Execute pipeline register. When asserted, *stall_d* and *stall_f* maintain the current values in the Decode and Fetch stage pipeline registers, respectively.

Control hazards necessitate flushing instructions from the pipeline registers in the Decode and Execute stages, mandating synchronous clearing of the registers. Upon a branch being taken, *flush_d* and *flush_e* are asserted to flush the Decode

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:53:30 UTC from IEEE Xplore. Restrictions apply.

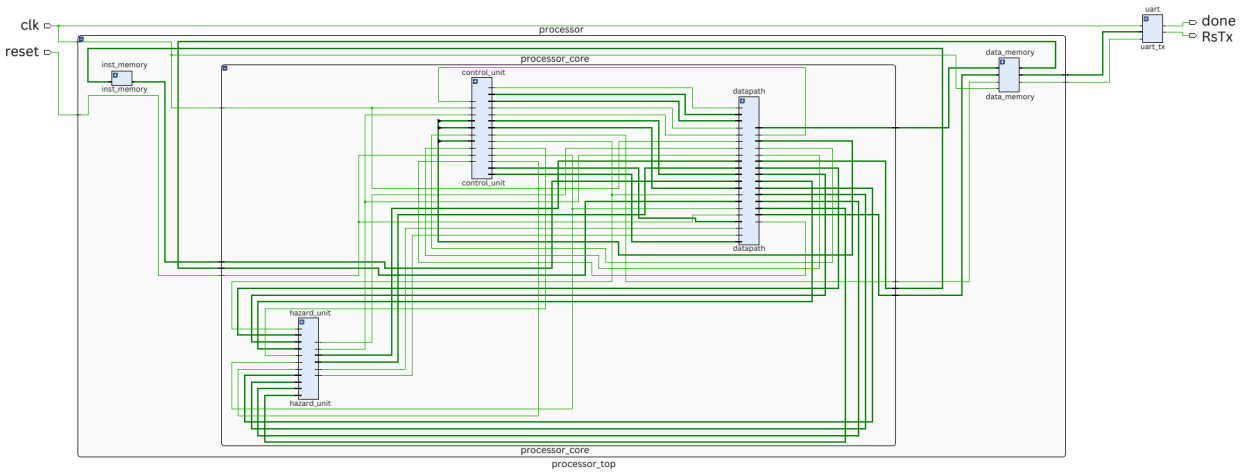


Fig. 3. RTL Schematic of the basic SoC

and Execute pipeline registers, introducing a bubble in the pipeline.

E. UART Periphral

For serial communication capabilities and to verify the functionality of the processor in the real world, the processor has been integrated with a UART peripheral with configurable baud rate to create a basic SoC. The peripheral was designed using a finite state machine (FSM), as described in [9], but has three states instead of four: READY, LOAD, and SEND. The memory map locations used by the processor to interact with the UART for data transmission are detailed below.

Location 100 in the Data Memory corresponds to the *uart_data* signal. This signal stores the data payload intended for transmission to an external device. The processor writes the data to this memory location before initiating the transmission process.

Location 104 in the Data Memory maps to the *uart_start* signal. This signal serves as the control signal for initiating data transmission. The processor writes the value 1 to this location to signal the UART to begin transmitting the data residing in the memory location 100. This write operation triggers the UART to commence serial transmission of the data, one bit at a time, over the designated communication line.

V. Experimental Work

The functionality of the Processor Core was verified using the Vivado Simulator. A memory file containing machine code was created to test all implemented RISC-V instructions, as specified in [10]. A testbench was written to exercise the processor with the memory file to verify its functionality. Successful execution would result in the processor storing the values 0x54 and 0x01 at memory locations 100 and 104 of the Data Memory, respectively. A snapshot of the simulation environment is included in Fig. 4.

The SoC design was then implemented on the Basys 3 Artix-7 FPGA. The machine code, designed to write the

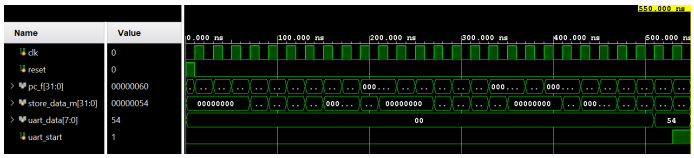


Fig. 4. Simulation of the Processor

uppercase alphabets (ASCII codes 0x41 to 0x5A) to memory location 100 sequentially, was loaded into the FPGA's memory. A serial terminal application (Tera Term) running on the computer was used to observe the output. The serial communication settings were set to a baud rate of 19200 and 1 stop bit. Transmission starts when 0x01 is written to memory location 104 after writing the first alphabet and stops upon writing 0x00 after all alphabets are sent. The experimental setup is shown in Fig. 5.

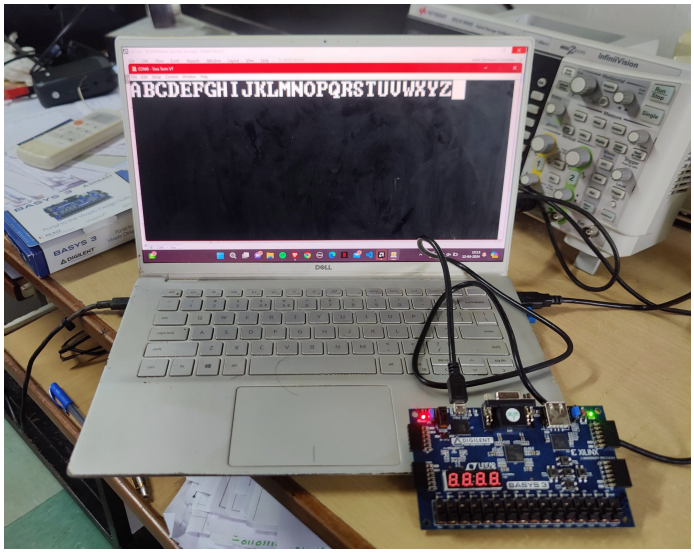


Fig. 5. Experimental Setup

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:53:30 UTC from IEEE Xplore. Restrictions apply.

VI. Result Analysis

The power reports and resource utilization of the basic SoC are given in Tables III and IV, respectively. The maximum clock frequency of the core was found to be 65 MHz, computed as:

$$F_{\text{MAX}}\ (\text{MHz}) = \max\left(\frac{1000}{T_i - \text{WNS}_i}\right) \tag{1}$$

TABLE III
Power Report

Power Report Details	Values
Total on Chip Power	0.112 W
Junction Temperature	25.6 C
Thermal Margin	59.4 C (11.8 W)
Dynamic Power	0.040 W
Device Static Power	0.072 W

TABLE IV
Utilization Report

Resource	Utilization	Available	% Usage
LUT	1994	20800	9.59
FF	3638	41600	8.75
DSP	12	90	13.33

To comprehensively assess the proposed core's performance, we juxtapose it with the core outlined in this paper and the core suggested in [4] and [8]. The core presented in this paper exhibits nearly a 15 MHz increase in speed to that presented in [4] (implements RV32IM ISA) and nearly a 3 MHz increase in speed to that presented in [8] (implements RV32I ISA).

TABLE V
Comparison Table

Resource	Utilization		
	Proposed Core	Core in [4]	Core in [8]
LUT	1289	3791	2115
FF	1542	2912	558
Total	2831	6703	2673

The proposed core also executes a greater number of instructions compared to the core suggested in [5], encompassing all six types of instructions (R-type, I-type, S-type, B-type, U-type and J-type) as opposed to four in the core discussed in [5] (R-type, I-type, S-type and B-type).

VII. Conclusion and Future Scope

This research focused on the design and implementation of a RISC-V processor. The primary objective was to create a core that could be efficiently implemented on FPGAs and help learners grasp the principles of RISC-V while offering a good balance between power consumption, resource utilization, and performance.

The implementation used the Vivado Design Suite 2023.2 and successfully targeted the Basys 3 Artix-7 FPGA. The processor leverages the growing RISC-V ecosystem, opening doors for future integration with existing toolchains and software designed for this ISA. This compatibility makes the processor highly reusable and adaptable for various RISC-V processor designs in the future.

The specific implementation resulted in a 5-stage pipelined, modular, and highly extensible processor core. The design exhibits efficient resource utilization on the chosen FPGA, achieving a peak clock speed of 65 MHz while maintaining a power consumption of 119 mW. This demonstrates the processor's potential for real-world applications where both performance and low power consumption are crucial.

There are exciting possibilities for further development of this core. Future enhancements could include the integration of an I/O bus, a multi-level cache mechanism, and support for interrupts. Additionally, incorporating a floating-point co-processor would expand the core's capabilities for applications requiring more complex calculations. These advancements are potential areas of focus for future research, to create an even more versatile and powerful RISC-V processor core suitable for a wider range of system applications.

References

[1] S. Harris and D. Harris, Digital Design and Computer Architecture-RISC-V Edition, Morgan Kaufmann, 2021

[2] Software Tools, RISC-V Foundation. [Online]. Available: https://riscv.org/software-tools/

[3] T. Zheng, G. Cai and Z. Huang, "A Soft RISC-V Processor IP with High-performance and Low-resource consumption for FPGA" 2022 IEEE International Symposium on Circuits and Systems (ISCAS), Austin, TX, USA, 2022, pp. 2538-2541.

[4] W. Zhang, Y. Zhang and K. Zhao, "Design and Verification of Three-stage Pipeline CPU Based on RISC-V Architecture" 2021 5th Asian Conference on Artificial Intelligence Technology (ACAIT), Haikou, China, 2021, pp. 697-703.

[5] M. H. Banna Saif, N. U. Sadad and M. N. Islam Mondal, "FPGA Implementation of Educational RISC- V Processor Suitable for Embedded Applications" 2023 International Conference on Electrical, Computer and Communication (ECCE), Chittagong, Bangladesh, 2023, pp. 1-5.

[6] J. Saussereau, C. Leroux, J. -B. Begueret and C. Jego, "AsteRISC: A Size-Optimized RISC-V Core for Design Space Exploration" 2023 IEEE International Symposium on Circuits and Systems (ISCAS), Monterey, CA, USA, 2023, pp. 1-5.

[7] I. Thanga Dharsni, K. S. Pande and M. K. Panda, "Optimized Hazard Free Pipelined Architecture Block for RV32I RISC-V Processor" 2022 3rd International Conference on Smart Electronics and Communication (ICOSEC), Trichy, India, 2022, pp. 739-746.

[8] A. E. Phangestu, I. T. Mujiono, M. I. Kom and S. Ahmad Zaini, "Five-Stage Pipelined 32-Bit RISC-V Base Integer Instruction Set Architecture Soft Microprocessor Core in VHDL" 2022 International Seminar on Intelligent Technology and Its Applications (ISITIA), Surabaya, Indonesia, 2022, pp. 304-309.

[9] A. K. Gupta, A. Raman, N. Kumar and R. Ranjan, "Design and Implementation of High-Speed Universal Asynchronous Receiver and Transmitter (UART)" 2020 7th International Conference on Signal Processing and Integrated Networks (SPIN), Noida, India, 2020, pp. 295-300.

[10] "The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Document Version 2.2", Editors Andrew Waterman and Krste Asanovi´c, RISC-V Foundation, May 2017.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:53:30 UTC from IEEE Xplore. Restrictions apply.